LINEAR REGRESSION PROJECT - AIML BATCH

Predict New York City Airbnb Prices Using Linear Regression

A beginner-friendly machine-learning project on 48,895 real Airbnb listings

Student name	Pallavi
Batch	October A
Submission date	30 October 2026
Project type	Individual project - Unlox AIML Batch assignment
Dataset source	New York City Airbnb Open Data (AB_NYC_2019), Kaggle. Copy used: github.com/4GeeksAcademy/data-preprocessing-project-tutorial (file AB_NYC_2019.csv).
Tools used	Python, Pandas, NumPy, Matplotlib, Seaborn, Scikit-learn

This report contains the full analysis, charts, results and (in the appendix) the complete Python code.

One-Page Summary (for a non-technical reader)

What was done. Using 48,895 Airbnb listings in New York City, I built a simple model that guesses the price per night of a listing from its borough, room type, location and a few other details. Extremely expensive listings and very long-stay listings were removed first, so the model focuses on the typical traveller market (nightly price up to $355 and minimum stay up to 30 nights).

R-squared = 0.487. In plain English: the model understands about **48.7%** of the reasons why prices differ from one listing to another. The remaining 51.3% is not captured by the model: it depends on things the dataset does not contain (such as number of bedrooms, amenities, photos or quality) and on effects a simple straight-line model cannot represent.

Mean Absolute Error (MAE) = $38.32. In plain English: on listings the model had never seen, its price guess is wrong by about **$38 per night on average**. The average price in the test listings is $123.67, so a typical mistake is about 31% of the price. For comparison, simply guessing the average price for every listing would be wrong by $58.52 on average, so the model is clearly better than a blind guess.

Where it works and where it does not. The model is reasonable for everyday listings, but it struggles with the most expensive ones: for listings priced $251-355 it underestimates by about $128 per night on average, because it cannot see the luxury features that make them expensive.

Bottom line. The model is a useful rough guide to a fair nightly price, not an exact price calculator.

1. Project Introduction and Objective

Airbnb is an online marketplace where hosts list rooms and homes for short stays. Setting the right nightly price is difficult. If a host asks too much the listing stays empty; if too little the host loses money. Machine learning can learn from thousands of existing listings and suggest a sensible price.

Objective. Build a complete, beginner-friendly **Linear Regression** model that predicts the nightly price of a New York City Airbnb listing, and judge honestly how good it is. The project follows the seven steps of the assignment:

- Step 1 - Exploratory Data Analysis and data auditing
- Step 2 - Data cleaning and missing values
- Step 3 - Outlier identification and treatment (95th percentile rule)
- Step 4 - Feature selection and categorical (One-Hot) encoding
- Step 5 - Feature-target separation and 80/20 train-test split
- Step 6 - Model training and evaluation (R-squared, MAE, MSE, RMSE)
- Step 7 - Residual analysis and interpretation (cheap vs expensive listings)

Key terms in simple words. *Target variable* = the thing we want to predict (price). *Features* = the facts we use to predict it (borough, room type, location, reviews and so on). *Linear Regression* = a method that finds the best straight-line style formula: price = a base amount plus (a number times each feature), added together.

2. Dataset Description and Source

Dataset: New York City Airbnb Open Data (file AB_NYC_2019.csv), the classic dataset named in the assignment, available on Kaggle. Kaggle itself could not be accessed from the environment used to run this analysis, so I downloaded the same file from a public GitHub copy (github.com/4GeeksAcademy/data-preprocessing-project-tutorial). The file has **48,895 rows** (one per listing, matching the 'around 49,000' in the assignment) and **16 columns**. No data was invented or changed by hand.

Columns and data types

Column	Data type	Unique values	Meaning
id	int64	48,895	Unique listing number
name	text (object)	47,905	Listing title
host_id	int64	37,457	Unique host number
host_name	text (object)	11,452	Host first name
neighbourhood_group	text (object)	5	Borough (5 values)
neighbourhood	text (object)	221	Specific neighbourhood (221 values)
latitude	float64	19,048	Location (north-south)
longitude	float64	14,718	Location (east-west)
room_type	text (object)	3	Entire home/apt, Private room or Shared room
price	int64	674	Price per night in US dollars - **TARGET**
minimum_nights	int64	109	Minimum stay required (nights)
number_of_reviews	int64	394	Total reviews received
last_review	text (object)	1,764	Date of the latest review
reviews_per_month	float64	937	Average reviews per month
calculated_host_listings_count	int64	47	Number of listings the host has
availability_365	int64	366	Days per year the listing is available

Sample records (first 5 rows, selected columns)

neighbourhood_group	room_type	price	min. nights	reviews	reviews/month	availability
Brooklyn	Private room	149	1	9	0.21	365
Manhattan	Entire home/apt	225	1	45	0.38	355
Manhattan	Private room	150	3	0	nan	365
Brooklyn	Entire home/apt	89	1	270	4.64	194
Manhattan	Entire home/apt	80	10	9	0.1	0

3. Exploratory Data Analysis (Step 1)

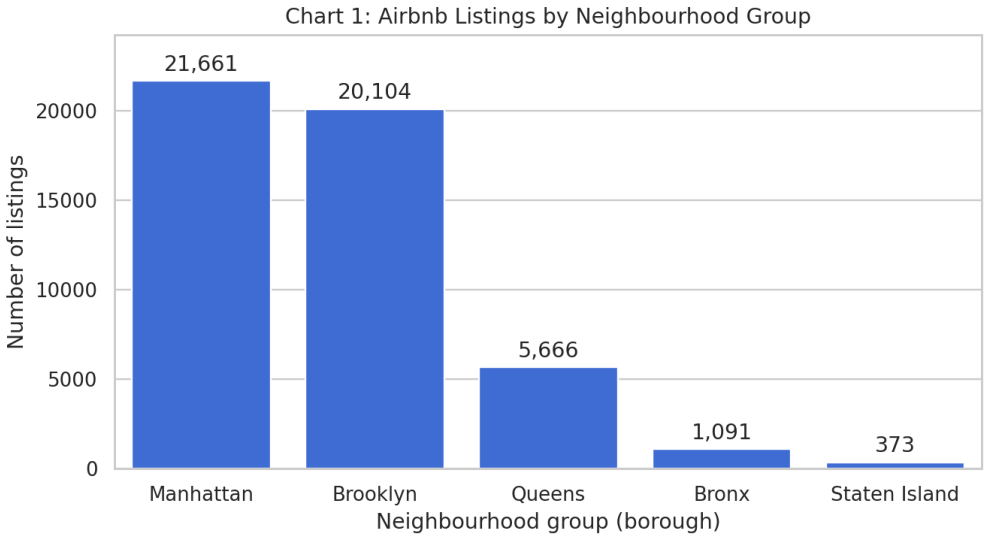
Task 1.1 - Shape, columns, data types. The raw data has 48,895 rows and 16 columns (see the table above). Numbers are stored as integers or decimals and text columns as text; the date column last_review is stored as text and not as a real date.

Task 1.2 - Types of variables.

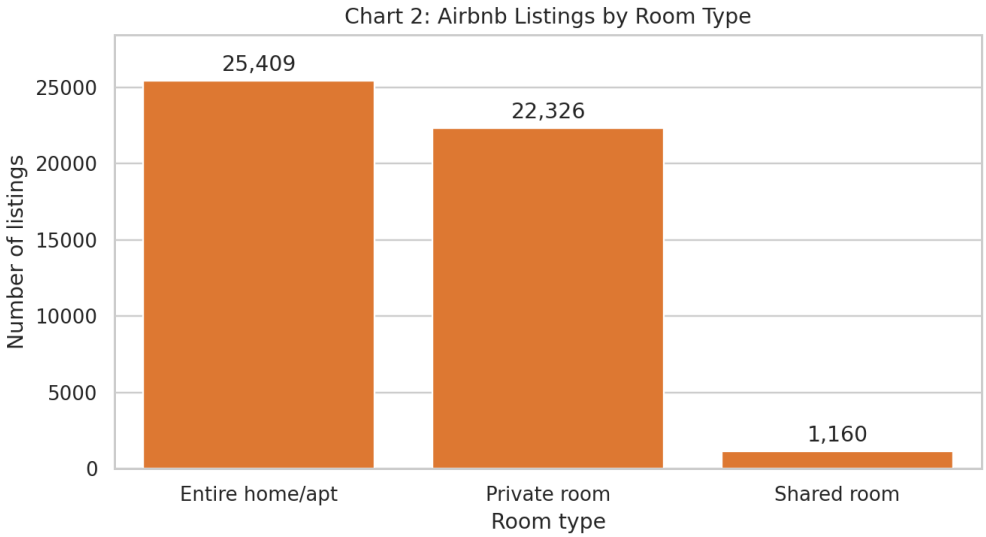
Kind of variable	Columns	Why
Numerical - discrete (whole-number counts)	minimum_nights, number_of_reviews, calculated_host_listings_count, availability_365	Counts of nights, reviews, listings and days
Numerical - continuous (any value)	price, latitude, longitude, reviews_per_month	Can take any decimal value; price is the target
Categorical - nominal (no order)	neighbourhood_group, neighbourhood, room_type (also name, host_name as free text)	Labels with no natural ranking
Categorical - ordinal (ordered)	None	No column has categories with a natural order
Identifiers	id, host_id	Just ID numbers; they carry no meaning for prediction

Key facts from the raw data

- Manhattan (44.3%) and Brooklyn (41.1%) hold 85.4% of all listings; Staten Island has only 0.76%.
- Entire homes/apartments are 52.0% of listings, private rooms 45.7% and shared rooms only 2.4%.
- Median price by borough: Manhattan $150, Brooklyn $90, Queens $75, Staten Island $75, Bronx $65.
- Median price by room type: entire home/apt $160, private room $70, shared room $45.



Explanation: Manhattan has 21,661 listings and Brooklyn 20,104. Queens has 5,666, the Bronx 1,091 and Staten Island only 373. The market is concentrated in two boroughs, so the model will learn most from Manhattan and Brooklyn listings.



Explanation: Entire homes/apartments (25,409) and private rooms (22,326) are almost equally common, while shared rooms are rare (1,160). Room type matters for price because a whole apartment costs much more than a room or a bed.

4. Data Auditing and Missing-Value Analysis (Step 1, Task 1.3)

Column	Missing values	Missing %	Comment
last_review	10,052	20.56%	Date text; missing for listings never reviewed
reviews_per_month	10,052	20.56%	Missing for listings never reviewed
name	16	0.03%	Listing title (not used by the model)
host_name	21	0.04%	Host name (not used by the model)
All other 12 columns	0	0.00%	Complete

Two columns, last_review and reviews_per_month, are each missing in 10,052 rows (20.56% of the data). The other missing values are tiny and are in columns that are dropped anyway. No duplicate rows were found after cleaning. The price column has no missing values, but 11 listings have a price of $0, which is checked in the next step.

5. Data Cleaning and Missing Values (Step 2)

Task	Investigation (actual result)	Decision and reason
2.1 last_review	Date text, 10,052 missing. Its missing rows are exactly the same rows as the missing reviews_per_month (yes).	**Dropped.** The only extra information in a date is how recent the review is; number_of_reviews and reviews_per_month already describe review activity. Turning the date into 'days since last review' would still leave 20% empty and needs an arbitrary reference date.
2.2 reviews_per_month	All 10,052 missing rows have number_of_reviews = 0 (min 0, max 0). Total listings with 0 reviews: 10,052, and 0 of them have a value present.	**Filled with 0.** The values are not random: a listing with no reviews has no reviews per month. Filling with the mean would be wrong.
2.3 Identifier columns	id, host_id (numbers) and name, host_name (free text)	**Dropped.** They only identify a listing or host. A formula cannot learn a pattern from a unique ID.
Extra check: price = 0	11 listings have price $0	**Removed.** A free night is not a genuine listing and would distort the target.

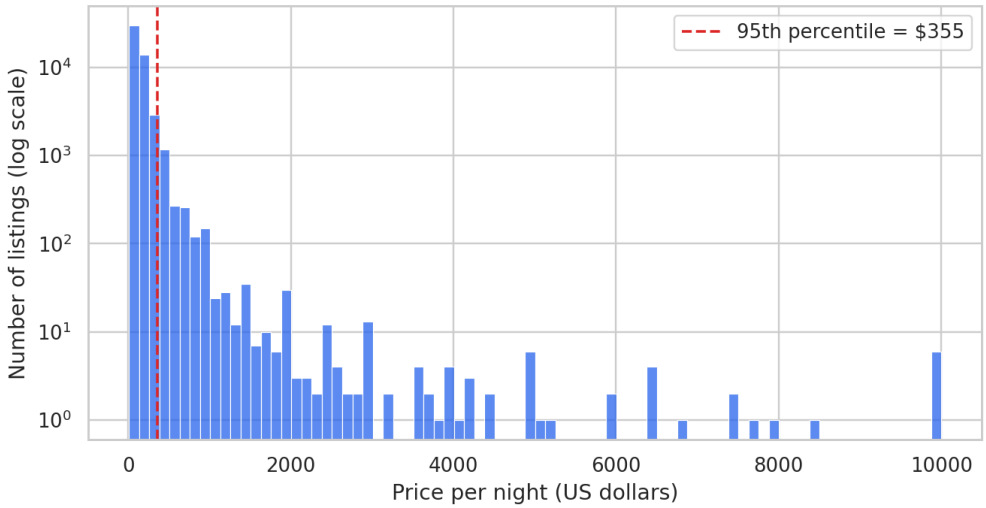
After cleaning: 48,884 rows and 11 columns, with **0 missing values** (started with 48,895 rows and 16 columns).

In simple words: the missing review values were not mistakes; they simply mean 'no reviews yet', so 0 is the logical value.

6. Outlier Identification and Treatment (Step 3)

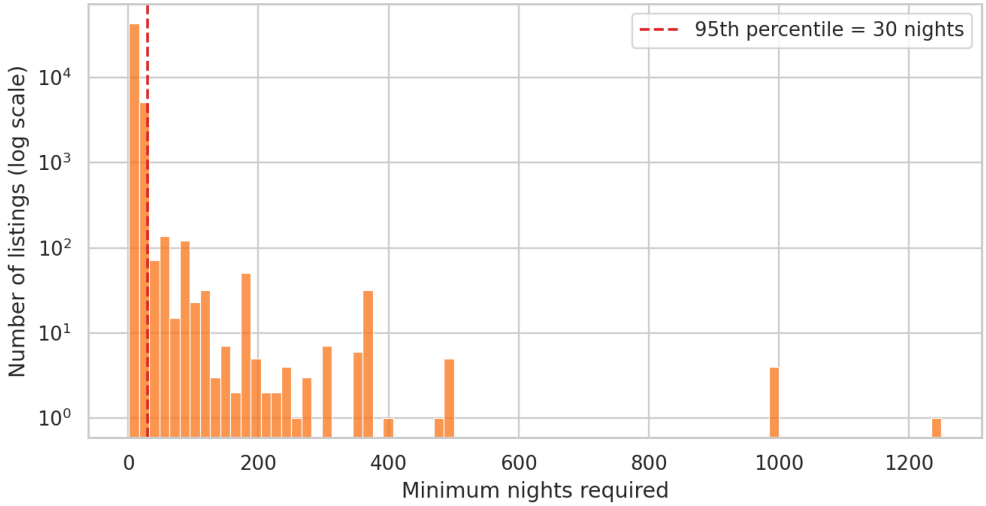
An outlier is a value far away from the rest. Linear Regression tries to reduce the squared error, so a few extreme values can drag the whole fitted line towards them. Price and minimum_nights were inspected first.

Chart 3: Distribution of Nightly Price (before outlier treatment)



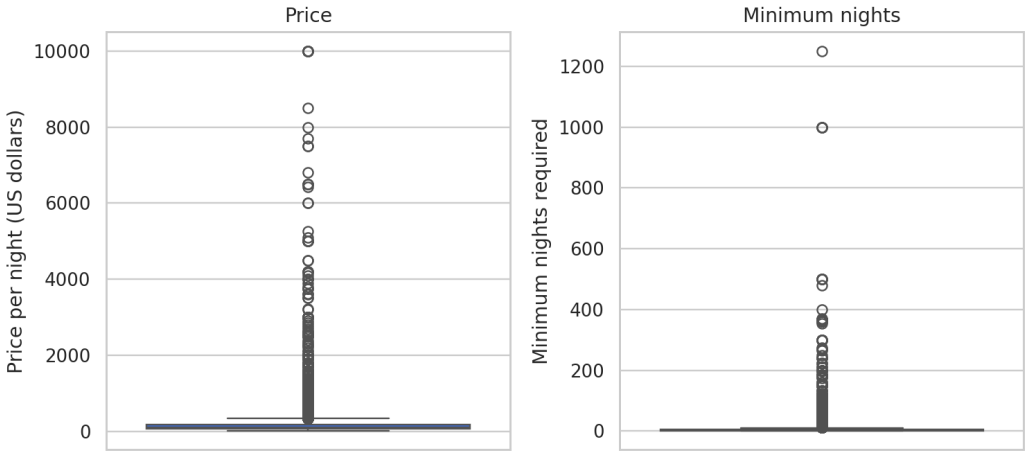
Explanation: Almost all listings are cheap, but a long thin tail reaches $10,000 per night (the y-axis is a log scale so the tail is visible). The 95th percentile is $355; 239 listings cost more than $1,000 and 3 cost $10,000.

Chart 4: Distribution of Minimum Nights (before outlier treatment)



Explanation: Most listings need only a few nights, but some demand months or years: the maximum is 1,250.0 nights. The 95th percentile is 30 nights; 43 listings require 365 or more nights, which are long-term rentals, not holiday stays.

Chart 5: Boxplots of Price and Minimum Nights (before treatment)



Explanation: The boxes are squashed flat at the bottom and a long column of dots (outliers) rises above them. By the IQR rule 2,972 prices and 6,605 minimum-night values are flagged. The mean price ($152.76) is far above the median ($106) because of the extreme values.

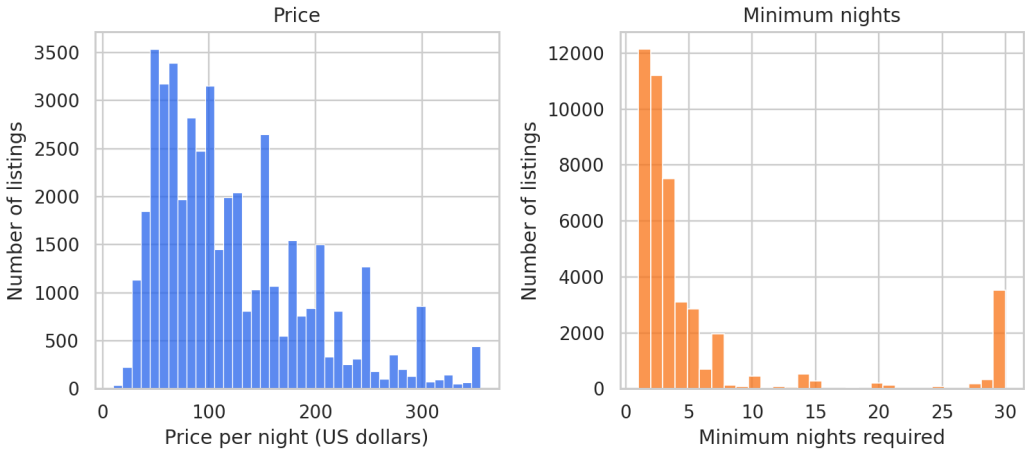
Treatment: 95th percentile rule (trimming)

I kept only listings with **price at or below the 95th percentile ($355)** and **minimum_nights at or below the 95th percentile (30 nights)**. This removes luxury penthouses (such as the $10,000 listings) and long-term rentals (such as stays of 365+ nights), so the model focuses on the typical consumer market, as the assignment asks. This removed 3,136 rows (6.4%): 2,441 for high price and 747 for long minimum stay (some rows break both rules). 45,748 rows remain.

Effect of outlier treatment

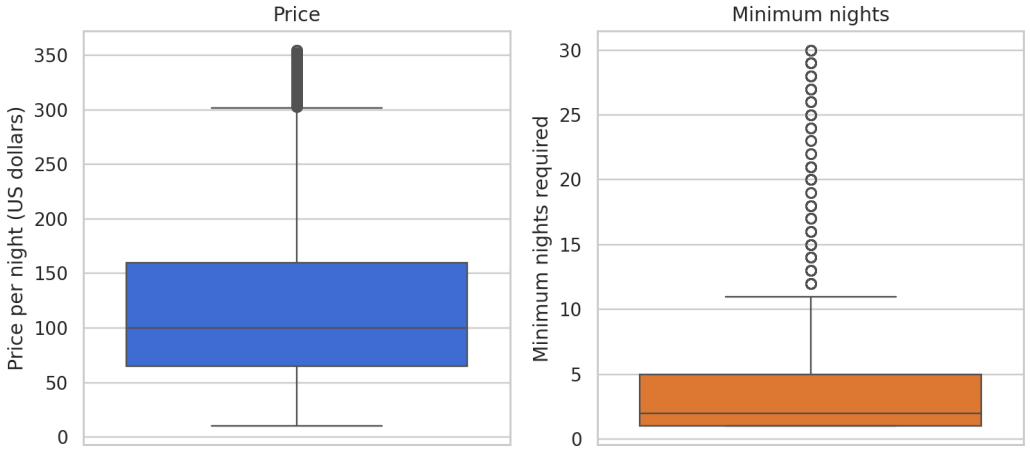
Measure	Before treatment	After treatment
Rows	48,884	45,748
Price: mean	$152.76	$122.54
Price: median	$106.00	$100.00
Price: standard deviation	$240.17	$71.95
Price: maximum	$10,000.00	$355.00
Minimum nights: mean	7.03	5.56
Minimum nights: standard deviation	20.51	8.18
Minimum nights: maximum	1,250.0	30.0

Chart 6: Price and Minimum Nights after Outlier Treatment



Explanation: After trimming, the price histogram shows a clear shape between about $10 and $355, with most listings around $100. Minimum nights are now 1 to 30, with a small bump at 30 (monthly stays).

Chart 7: Boxplots of Price and Minimum Nights (after treatment)



Explanation: The boxes are now readable. The price spread (standard deviation) fell from $240 to $72 and the maximum minimum-stay fell from 1,250.0 to 30 nights. A few dots remain (893 prices by the IQR rule), but they are ordinary high-priced listings, so they were kept.

Important consequence: the final model is only valid for listings up to $355 per night and 30 nights minimum stay. It should not be used to price luxury or long-term rentals.

7. Feature Selection and Categorical Encoding (Step 4)

Task 4.1 - High cardinality. *Cardinality* means the number of different values in a column. neighbourhood has 221 different values in the raw data (219 remain after cleaning), while neighbourhood_group has only 5.

A computer cannot do maths on words, so each category must become 0/1 columns. Encoding neighbourhood would create about 218 new columns, but encoding neighbourhood_group creates only 4. This is the **Curse of Dimensionality**: with so many columns, each neighbourhood has few listings, so the model learns from very little data per column, becomes slow and complicated, can memorise noise (overfitting) and becomes hard to explain. Many neighbourhoods have only a handful of listings. Latitude and longitude already describe location precisely, and neighbourhood_group gives the broad area. **Decision: the neighbourhood column was dropped.**

Task 4.2 - One-Hot Encoding with drop_first=True. neighbourhood_group and room_type were converted into 0/1 columns using pandas get_dummies with drop_first=True. Dropping the first category avoids the **Dummy Variable Trap**: if you keep a column for every category, any one column can be worked out from the others (for example, if a listing is not Private room and not Shared room, it must be Entire home/apt). That perfect redundancy confuses a linear model. The dropped (reference) categories are **Bronx** and **Entire home/apt**.

Original column	Categories	New columns created (drop_first=True)
neighbourhood_group	Bronx, Brooklyn, Manhattan, Queens, Staten Island (5)	neighbourhood_group_Brooklyn, _Manhattan, _Queens, _Staten Island (4); Bronx is the reference
room_type	Entire home/apt, Private room, Shared room (3)	room_type_Private room, room_type_Shared room (2); Entire home/apt is the reference

After encoding the data has 45,748 rows and 14 columns: price (target) plus 13 features:

latitude, longitude, minimum_nights, number_of_reviews, reviews_per_month, calculated_host_listings_count, availability_365, neighbourhood_group_Brooklyn, neighbourhood_group_Manhattan, neighbourhood_group_Queens, neighbourhood_group_Staten Island, room_type_Private room, room_type_Shared room.

8. Feature-Target Separation and Training/Testing Split (Step 5)

Task 5.1. X (features) contains all columns except price; **y** (target) is the price column.

Task 5.2. The data was split into 80% training and 20% testing using train_test_split(test_size=0.20, random_state=42). Training set: **36,598** listings. Testing set: **9,150** listings. Average price: $122.26 (training) and $123.67 (testing), so both parts look alike.

Why split? A student who sees the exam questions in advance can memorise them. The model is the same: if we test it on the data it learned from, the score looks better than reality. The test set is the 'hidden final exam' that shows how it behaves on listings it has never seen. **Why random_state=42?** It fixes the random shuffling, so anyone running the code gets exactly the same split and the same results.

9. Linear Regression Model Implementation (Step 6)

Using scikit-learn: **model = LinearRegression()** creates the model, **model.fit(X_train, y_train)** trains it on the training data, and **model.predict(X_test)** predicts prices for the hidden test listings. The model learns one number (coefficient) for every feature and one starting number (intercept) so that predicted price = intercept + sum(coefficient x feature).

Learned coefficients (largest first)

Feature	Coefficient	Plain-English meaning
longitude	-297.507	Location effect per degree of longitude (hard to read on its own; works together with latitude)
room_type_Shared room	-104.577	A shared room is about this much cheaper per night than an entire home, other things equal
neighbourhood_group_Staten Island	-82.934	Staten Island is about this much cheaper than the Bronx, other things equal
room_type_Private room	-79.117	A private room is about this much cheaper than an entire home, other things equal
latitude	-76.227	Location effect per degree of latitude (read together with longitude)
neighbourhood_group_Manhattan	23.448	Manhattan is about this much dearer than the Bronx, other things equal
neighbourhood_group_Brooklyn	-8.211	Brooklyn is slightly cheaper than the Bronx, other things equal
neighbourhood_group_Queens	5.668	Queens is slightly dearer than the Bronx, other things equal
reviews_per_month	-1.060	Small effect per extra review per month
minimum_nights	-1.038	Each extra required night lowers the price slightly
calculated_host_listings_count	0.137	Tiny effect per extra listing of the host
availability_365	0.075	Tiny effect per extra available day
number_of_reviews	-0.068	Tiny effect per extra review

Intercept = -18,740.60. The intercept looks huge only because longitude and latitude are large raw numbers that it balances out; it has no direct meaning. Coefficients show associations in this dataset, not proof of cause. The features were not rescaled, so the sizes of coefficients for different kinds of features should not be compared directly, except for the 0/1 columns, which are all on the same scale.

10. Model Evaluation (Step 6, Task 6.3)

All four metrics were calculated on the 9,150 hidden test listings.

Metric	Test result	Training result	What it means in simple words
R-squared (R2)	0.4868	0.4749	The model explains about 48.7% of the differences in price. 1.0 would be perfect; 0 means no better than always guessing the average.
Mean Absolute Error (MAE)	$38.32	$37.86	On average the guess is $38 away from the real price per night.
Mean Squared Error (MSE)	2,760.62 (dollars squared)	2,691.58	Average of the squared mistakes. Squaring makes big mistakes count much more, so MSE punishes large errors heavily. Its unit is dollars squared, which is hard to picture.
Root Mean Squared Error (RMSE)	$52.54	$51.88	The square root of MSE, back in dollars. Because it is larger than MAE ($52.54 vs $38.32), some mistakes are much bigger than the average mistake.

How good is this? Three checks

- **Compared with a blind guess.** Always guessing the average training price gives MAE $58.52 and RMSE $73.36. The model improves these to $38.32 and $52.54.
- **Overfitting check.** Training R2 (0.475) and test R2 (0.487) are very close, so the model is not memorising the training data.
- **Size of the error.** An average mistake of $38 on an average price of $124 is about 31%. That is useful as a rough guide but not precise.

Conclusion on R2: a value of 0.49 is moderate. Price depends on many things the dataset does not contain (bedrooms, bathrooms, amenities, building quality, photos), and a straight-line model also cannot capture

complicated location effects.

11. Summary of Visualizations

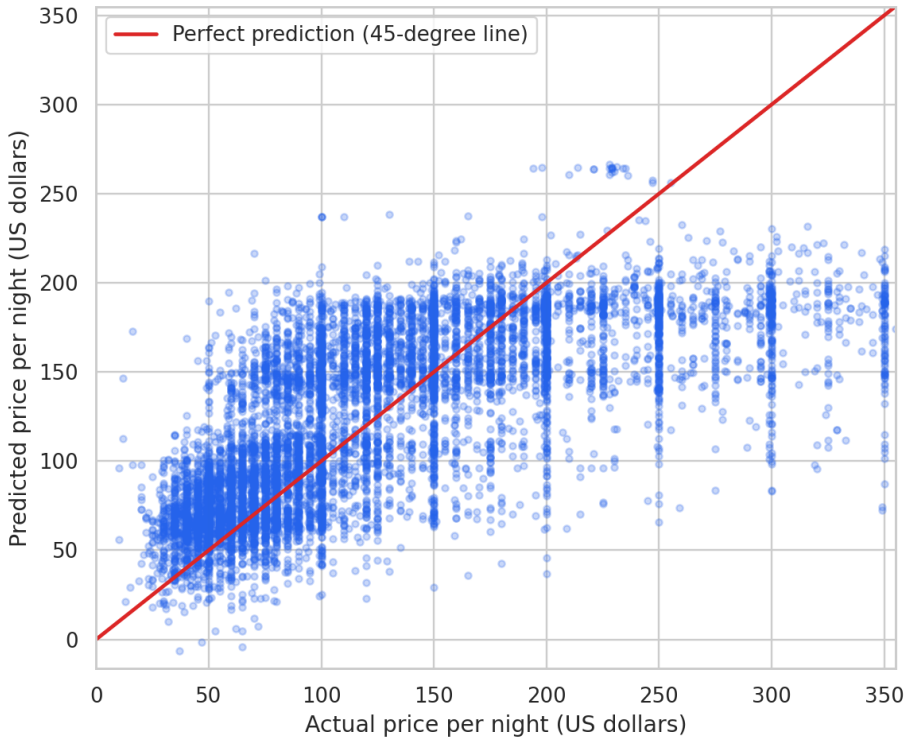
All charts have titles, axis labels and an explanation next to them. They appear in the section where they are used:

Chart	Type	Required by assignment	Section
1 Listings by neighbourhood group	Bar	Listings by neighbourhood group	3. EDA
2 Listings by room type	Bar	Listings by room type	3. EDA
3 Price distribution	Histogram	Price distribution histogram	6. Outliers
4 Minimum nights distribution	Histogram	Minimum nights distribution	6. Outliers
5 and 7 Price and minimum nights boxplots (before / after)	Boxplot	Price and minimum nights boxplots	6. Outliers
6 Histograms after treatment	Histogram	Effect of outlier treatment	6. Outliers
8 Actual vs predicted prices	Scatter + 45-degree line	Actual vs predicted	12. Residual analysis
9 Residual plot and 10 error histogram	Scatter / histogram	Residual analysis	12. Residual analysis
11 Error by price band	Grouped bar	Cheap vs expensive question	12. Residual analysis

12. Residual Analysis and Interpretation (Step 7)

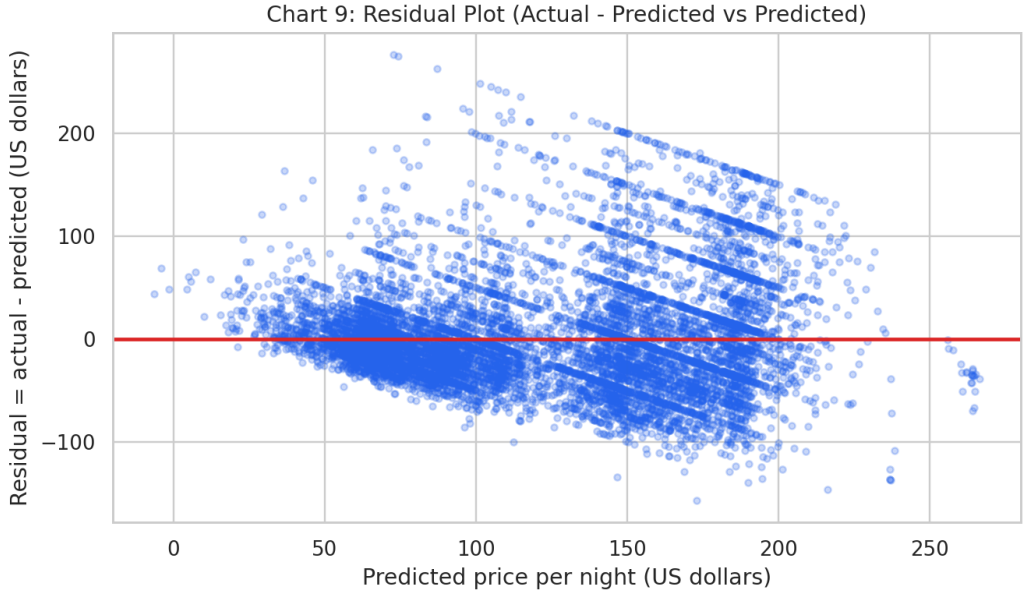
A **residual** is the prediction error for one listing: **actual price minus predicted price**. A positive residual means the model guessed too low; a negative residual means it guessed too high.

Chart 8: Actual vs Predicted Prices (test set)



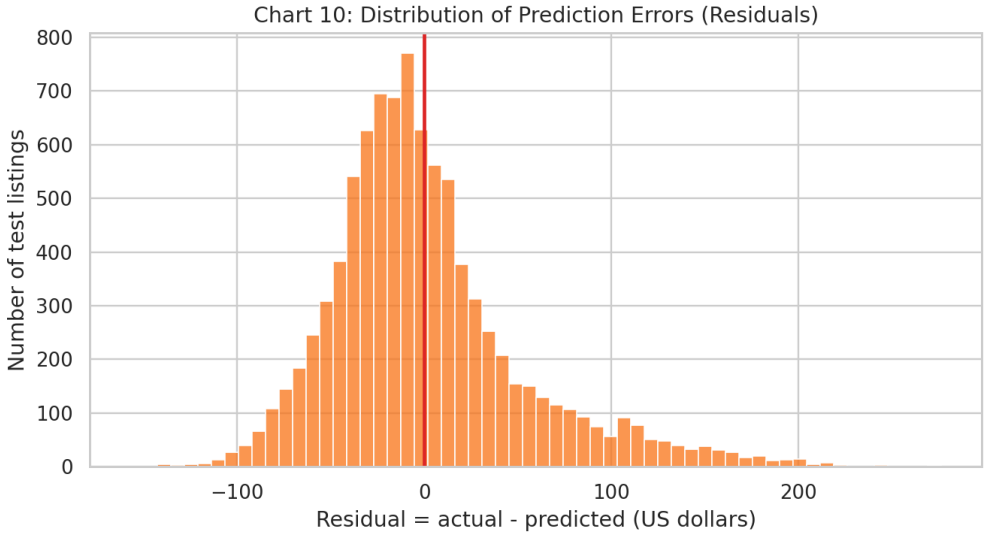
Explanation: Each dot is one test listing. The red 45-degree line is a perfect prediction (predicted = actual). Dots below the line were predicted too low, dots above too high. For cheaper listings the dots follow the line quite well, but as the actual price rises the dots flatten out: the highest prediction is only $267 although actual prices reach $355. The model compresses expensive listings towards the middle. 3 predictions were slightly negative (lowest $-6.47), which is impossible for a real price and shows a limit of a straight-line model.

Chart 9: Residual Plot (Actual - Predicted vs Predicted)



Explanation: If the model were ideal, the dots would form an even cloud around the red zero line. Here the cloud tilts: for low predicted prices the errors are mostly slightly negative, and for high predicted prices there are many large positive errors (actual price far above predicted). The diagonal stripes appear because hosts choose round prices such as $100, $150 or $200. Errors range from $-157 to +$276.

Chart 10: Distribution of Prediction Errors (Residuals)

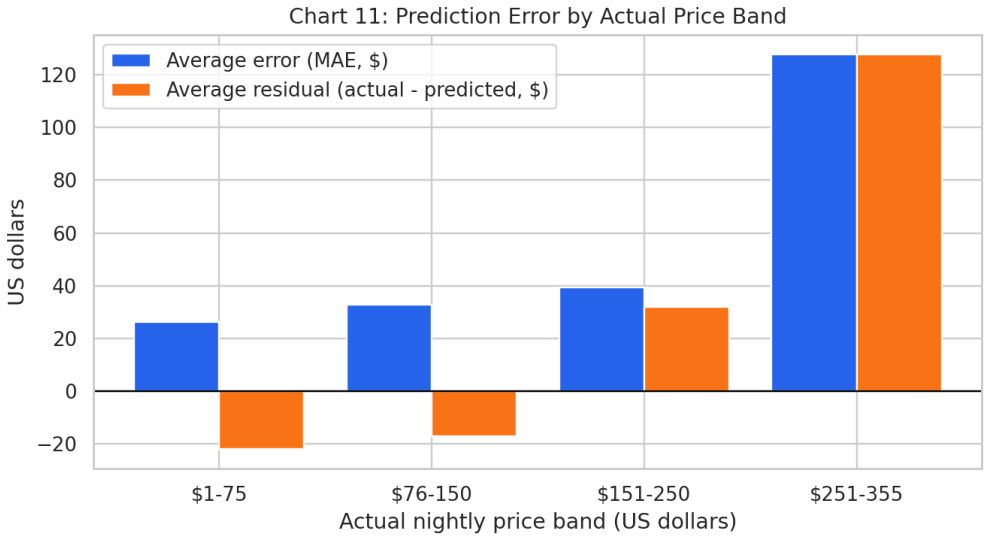


Explanation: The errors are centred close to zero (average residual $0.98), so the model is not biased overall. 45.2% of test predictions are within $25 of the real price and 73.7% are within $50. The right-hand tail is longer than the left: there are more big under-predictions (expensive listings guessed too low) than big over-predictions.

Does the model perform better on cheap or expensive listings, and why?

Test listings were grouped by their **actual** price and the errors were measured in each group:

Actual price band	Listings	Avg actual	Avg predicted	MAE ($)	Avg residual ($)	Median % error
$1-75	2,990	$55.14	$77.19	26.22	-22.06	38.8%
$76-150	3,665	$111.44	$128.67	32.78	-17.23	25.4%
$151-250	1,884	$198.20	$166.36	39.28	+31.84	15.9%
$251-355	611	$302.64	$174.85	127.80	+127.79	40.6%



Explanation: Blue bars (average size of the mistake) grow with price: $26 for the cheapest band to $128 for the dearest. Orange bars show the direction: negative for cheap listings (guessed too high) and strongly positive for the most expensive listings (guessed too low).

Answer. It depends on how the error is measured, and both views are true:

- **In dollars, the model is better on cheap listings.** The average mistake is $26 for listings of $1-75 but $128 for listings of $251-355, about 4.9 times larger.
- **In percentage terms, it is best for mid-to-upper listings ($151-250, median error 15.9%)** and weakest at both ends: 38.8% for $1-75 and 40.6% for $251-355. A $26 mistake is large when the real price is $55.
- **The direction of the mistakes differs.** Cheap listings are over-predicted (average $22 too high for $1-75), while the dearest listings are badly under-predicted (average $128 too low for $251-355; average prediction $175 versus actual $303). Overall, the model is more reliable for typical, lower-priced listings and unreliable for the high end.

Why does this happen?

- Linear Regression fits one straight-line formula and, to keep the *average* error small, it pulls every prediction towards the middle price. Cheap listings get pulled up and expensive ones pulled down (a pattern called regression towards the mean).
- What makes a listing truly expensive (size, number of bedrooms, luxury features, exact building and view) is **not in the dataset**. Two apartments in the same borough, room type and location can have very different prices, and the model cannot tell them apart.
- Squared-error training means a few very large errors at the top end dominate; this is why RMSE ($52.54) is clearly larger than MAE ($38.32).
- Price depends on location in a curved, patchy way (a few blocks can change prices a lot), whereas the model uses latitude and longitude as straight-line effects.

Extra check: errors by room type

Room type	Test listings	MAE ($)	Average residual ($)
Entire home/apt	4,591	50.27	+2.77
Private room	4,350	26.24	-0.69
Shared room	209	26.98	-3.71

Entire homes/apartments, which cover the wide range of higher prices, have a much larger average error than private or shared rooms, which supports the explanation above.

13. Final Findings and Interpretation

1. **Real, large dataset.** 48,895 listings; 85.4% are in Manhattan or Brooklyn and 97.6% are entire homes or private rooms.
2. **Missing reviews are structural.** All 10,052 missing reviews_per_month rows have zero reviews, so filling with 0 is logical; last_review was dropped.
3. **Outliers were extreme.** Prices up to $10,000 and minimum stays up to 1,250.0 nights. Trimming at the 95th percentile removed 6.4% of rows and cut the price standard deviation from $240 to $72.
4. **Room type and borough matter most.** Compared with an entire home, a private room is $79 cheaper and a shared room $105 cheaper per night; Manhattan is $23 dearer than the Bronx (other things equal).
5. **Model performance.** R2 = 0.4868, MAE = $38.32, MSE = 2,760.62, RMSE = $52.54 on 9,150 unseen listings, clearly better than guessing the average (MAE $58.52) and not overfitted (training R2 0.475).
6. **Review-related features have almost no effect.** number_of_reviews, availability_365 and calculated_host_listings_count have coefficients close to zero.
7. **Cheap vs expensive.** Errors are smaller in dollars on cheap listings ($26 vs $128), cheap ones are over-predicted and the most expensive ones are under-predicted by about $128.

14. Conclusion and Limitations

A simple Linear Regression model, trained on 36,598 listings, predicts the nightly price of a typical New York City Airbnb with R2 = 0.49 and an average error of $38. It captures the main price drivers (room type and area), but cannot explain luxury pricing, so it is best seen as a transparent starting point.

Limitations

- **Valid only for typical listings.** The model was built after removing listings above $355 per night or 30 minimum nights, so it should not be used for luxury or long-term rentals.
- **Missing important features.** No bedrooms, bathrooms, amenities, photos, host rating or seasonality in the data.
- **Linear assumptions.** A straight-line model cannot capture curved or patchy effects such as precise location, and it can predict slightly negative prices (3 test cases).
- **Listed prices and a single snapshot.** Prices are listed (asked) prices from one dataset (named 2019); today's prices will differ.
- **One split only.** Results come from one random 80/20 split (random_state=42); a different split would give slightly different numbers.
- **Trimming changes the question.** The 95th percentile rule is a judgement call; a different cut-off would give different scores.

Possible future improvements

Predict the logarithm of price to reduce the pull of expensive listings; try neighbourhood-level information using grouping or target encoding; add features such as bedrooms if available; compare with Ridge/Lasso regression, Random Forest or Gradient Boosting; and use cross-validation for more reliable scores.

15. References

- New York City Airbnb Open Data (AB_NYC_2019), Kaggle - dataset named in the assignment. https://www.kaggle.com/datasets (search: 'New York City Airbnb Open Data').
- Copy of AB_NYC_2019.csv used for this analysis: GitHub repository 4GeeksAcademy/data-preprocessing-project-tutorial - https://github.com/4GeeksAcademy/data-preprocessing-project-tutorial

- Unlox AIML Batch assignment: 'LINEAR REGRESSION-Project for AIML BATCH - Predict New York City Airbnb Prices' (task description).
- pandas documentation: https://pandas.pydata.org/docs/
- NumPy documentation: https://numpy.org/doc/
- Matplotlib documentation: https://matplotlib.org/stable/
- Seaborn documentation: https://seaborn.pydata.org/
- scikit-learn documentation (LinearRegression, train_test_split, metrics): https://scikit-learn.org/stable/

The analysis was written and run with the help of an AI assistant (Claude, by Anthropic). The complete code is in the appendix so it can be re-run and explained.

Appendix: Complete Python Source Code

This is the complete code that produced every number, table and chart in this report. It is organised in numbered steps matching the assignment. Lines starting with # are comments that explain each step in simple language. It can be run as a normal Python file or pasted cell by cell (each '# %%' line starts a new cell) into Jupyter Notebook or Google Colab. Install the libraries with: pip install pandas numpy matplotlib seaborn scikit-learn. The charts are saved in a folder named 'charts' and the calculated numbers in results.json.

```
# ======================================================================
# PREDICT NEW YORK CITY AIRBNB PRICES USING LINEAR REGRESSION  (Pallavi)
# Dataset : New York City Airbnb Open Data (AB_NYC_2019), 48,895 listings
# Tools   : Python, Pandas, NumPy, Matplotlib, Seaborn, Scikit-learn
# Works as a normal .py file or can be pasted cell by cell into Jupyter.
# ======================================================================

# %% STEP 0 - IMPORT LIBRARIES AND SETTINGS
import json
import os
import numpy as np                      # numbers and maths
import pandas as pd                     # tables (DataFrames)
import matplotlib
matplotlib.use("Agg")                   # draw charts without a window
import matplotlib.pyplot as plt         # charts
import seaborn as sns                   # nicer charts
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import (r2_score, mean_absolute_error,
                             mean_squared_error)

sns.set_theme(style="whitegrid", context="notebook")
BLUE, ORANGE, RED = "#2563eb", "#f97316", "#dc2626"
OUT_DIR = "charts"                      # folder for saved chart images
os.makedirs(OUT_DIR, exist_ok=True)
results = {}                            # every calculated number goes here


def save_chart(filename):
    """Tidy the layout and save the current chart as a PNG image."""
    plt.tight_layout()
    plt.savefig(os.path.join(OUT_DIR, filename), dpi=170)
    plt.close()


# %% STEP 1 - EXPLORATORY DATA ANALYSIS AND DATA AUDITING
# Task 1.1: load the data and look at shape, columns and data types
DATA_URL = ("https://raw.githubusercontent.com/4GeeksAcademy/"
            "data-preprocessing-project-tutorial/main/AB_NYC_2019.csv")
try:
    raw = pd.read_csv(DATA_URL)
except Exception:                       # fallback if there is no internet
    raw = pd.read_csv("AB_NYC_2019.csv")

df = raw.copy()
results["shape_raw"] = list(df.shape)
results["columns"] = list(df.columns)
results["dtypes"] = {c: str(t) for c, t in df.dtypes.items()}
results["sample_rows"] = df.head(5).astype(str).values.tolist()
results["n_unique"] = {c: int(df[c].nunique()) for c in df.columns}

# Task 1.2: numerical (discrete / continuous) and categorical variables
discrete_cols = ["minimum_nights", "number_of_reviews",
                 "calculated_host_listings_count", "availability_365"]
continuous_cols = ["price", "latitude", "longitude", "reviews_per_month"]
nominal_cols = ["neighbourhood_group", "neighbourhood", "room_type",
                "name", "host_name"]
identifier_cols = ["id", "host_id"]
results["discrete_cols"] = discrete_cols
results["continuous_cols"] = continuous_cols
results["nominal_cols"] = nominal_cols
results["identifier_cols"] = identifier_cols
# There is no true ordinal column (no category with a natural order).

# Task 1.3: missing values and their percentages
missing = df.isna().sum()
missing_pct = (missing / len(df) * 100).round(2)
results["missing_count"] = missing[missing > 0].to_dict()
results["missing_pct"] = missing_pct[missing > 0].to_dict()
results["describe_raw"] = df[["price", "minimum_nights", "number_of_reviews",
```

```
                            "reviews_per_month", "calculated_host_listings_count",
                            "availability_365"]].describe().T.round(2
                            ).reset_index().rename(
                            columns={"index": "column"}).to_dict("records")
results["share_group"] = (df["neighbourhood_group"].value_counts(
    normalize=True) * 100).round(2).to_dict()
results["count_group"] = df["neighbourhood_group"].value_counts().to_dict()
results["share_room"] = (df["room_type"].value_counts(
    normalize=True) * 100).round(2).to_dict()
results["count_room"] = df["room_type"].value_counts().to_dict()
results["median_price_group"] = df.groupby("neighbourhood_group")[
    "price"].median().to_dict()
results["median_price_room"] = df.groupby("room_type")[
    "price"].median().to_dict()

# Chart 1 - listings by neighbourhood group
order = df["neighbourhood_group"].value_counts().index
plt.figure(figsize=(8, 4.5))
ax = sns.countplot(data=df, x="neighbourhood_group", order=order, color=BLUE)
ax.bar_label(ax.containers[0], padding=3, fmt="{:,.0f}")
ax.set_ylim(0, df["neighbourhood_group"].value_counts().max() * 1.12)
plt.title("Chart 1: Airbnb Listings by Neighbourhood Group")
plt.xlabel("Neighbourhood group (borough)")
plt.ylabel("Number of listings")
save_chart("c1_groups.png")

# Chart 2 - listings by room type
order = df["room_type"].value_counts().index
plt.figure(figsize=(8, 4.5))
ax = sns.countplot(data=df, x="room_type", order=order, color=ORANGE)
ax.bar_label(ax.containers[0], padding=3, fmt="{:,.0f}")
ax.set_ylim(0, df["room_type"].value_counts().max() * 1.12)
plt.title("Chart 2: Airbnb Listings by Room Type")
plt.xlabel("Room type")
plt.ylabel("Number of listings")
save_chart("c2_room_types.png")

# %% STEP 2 - DATA CLEANING AND MISSING VALUES
# Task 2.1: last_review is a date text with many missing values.
# Are the missing last_review rows the same rows as missing reviews_per_month?
same_rows = bool((df["last_review"].isna() == df["reviews_per_month"].isna()).all())
results["last_review_same_missing_rows"] = same_rows

# Task 2.2: why is reviews_per_month missing? Check number_of_reviews there.
miss_rows = df[df["reviews_per_month"].isna()]
results["rpm_missing_rows"] = int(len(miss_rows))
results["rpm_missing_reviews_max"] = int(miss_rows["number_of_reviews"].max())
results["rpm_missing_reviews_min"] = int(miss_rows["number_of_reviews"].min())
results["zero_review_rows"] = int((df["number_of_reviews"] == 0).sum())
results["rpm_present_zero_reviews"] = int(((df["number_of_reviews"] == 0) &
                                           df["reviews_per_month"].notna()).sum())

# Decision: a listing with 0 reviews has 0 reviews per month, so fill with 0.
df["reviews_per_month"] = df["reviews_per_month"].fillna(0)
# Decision: drop last_review (a date text; its information is already
# captured by number_of_reviews and reviews_per_month).
df = df.drop(columns=["last_review"])

# Task 2.3: drop identifier / text columns that do not help the model
df = df.drop(columns=["id", "name", "host_id", "host_name"])

# Extra check: a price of 0 dollars per night is not a real listing
results["rows_price_zero"] = int((df["price"] == 0).sum())
df = df[df["price"] > 0].reset_index(drop=True)
results["missing_after_cleaning"] = int(df.isna().sum().sum())
results["shape_after_cleaning"] = list(df.shape)
results["duplicates_after_cleaning"] = int(df.duplicated().sum())

# %% STEP 3 - OUTLIER IDENTIFICATION AND TREATMENT
# Task 3.1: histograms and boxplots of price and minimum_nights (BEFORE)
def iqr_outliers(series):
    q1, q3 = series.quantile(0.25), series.quantile(0.75)
    iqr = q3 - q1
    return int(((series < q1 - 1.5 * iqr) | (series > q3 + 1.5 * iqr)).sum())

p95_price = float(df["price"].quantile(0.95))
p95_nights = float(df["minimum_nights"].quantile(0.95))
results["p95_price"], results["p95_nights"] = p95_price, p95_nights
results["p99_price"] = float(df["price"].quantile(0.99))
results["p99_nights"] = float(df["minimum_nights"].quantile(0.99))
```

```
results["iqr_outliers_before"] = {"price": iqr_outliers(df["price"]),
                                  "minimum_nights": iqr_outliers(df["minimum_nights"])}
results["price_over_1000"] = int((df["price"] > 1000).sum())
results["price_ge_10000"] = int((df["price"] >= 10000).sum())
results["nights_ge_365"] = int((df["minimum_nights"] >= 365).sum())
results["nights_gt_30"] = int((df["minimum_nights"] > 30).sum())
stats_before = df[["price", "minimum_nights"]].agg(
    ["mean", "median", "std", "max"]).round(2)
results["stats_before"] = stats_before.to_dict()
before_df = df.copy()

# Chart 3 - price histogram (BEFORE)
plt.figure(figsize=(8, 4.5))
sns.histplot(df["price"], bins=80, color=BLUE)
plt.yscale("log")
plt.axvline(p95_price, color=RED, ls="--", label=f"95th percentile = ${p95_price:,.0f}")
plt.title("Chart 3: Distribution of Nightly Price (before outlier treatment)")
plt.xlabel("Price per night (US dollars)")
plt.ylabel("Number of listings (log scale)")
plt.legend()
save_chart("c3_price_hist.png")

# Chart 4 - minimum nights histogram (BEFORE)
plt.figure(figsize=(8, 4.5))
sns.histplot(df["minimum_nights"], bins=80, color=ORANGE)
plt.yscale("log")
plt.axvline(p95_nights, color=RED, ls="--",
            label=f"95th percentile = {p95_nights:,.0f} nights")
plt.title("Chart 4: Distribution of Minimum Nights (before outlier treatment)")
plt.xlabel("Minimum nights required")
plt.ylabel("Number of listings (log scale)")
plt.legend()
save_chart("c4_nights_hist.png")

# Chart 5 - boxplots BEFORE treatment
fig, axes = plt.subplots(1, 2, figsize=(9, 4.5))
sns.boxplot(y=df["price"], ax=axes[0], color=BLUE)
axes[0].set_title("Price")
axes[0].set_ylabel("Price per night (US dollars)")
sns.boxplot(y=df["minimum_nights"], ax=axes[1], color=ORANGE)
axes[1].set_title("Minimum nights")
axes[1].set_ylabel("Minimum nights required")
fig.suptitle("Chart 5: Boxplots of Price and Minimum Nights (before treatment)")
save_chart("c5_boxplots_before.png")

# Task 3.2: 95th percentile rule - keep only listings at or below the
# 95th percentile for BOTH price and minimum_nights (trimming)
df = df[(df["price"] <= p95_price) &
        (df["minimum_nights"] <= p95_nights)].reset_index(drop=True)
results["shape_after_outliers"] = list(df.shape)
results["rows_removed_outliers"] = int(len(before_df) - len(df))
results["rows_removed_price_only"] = int((before_df["price"] > p95_price).sum())
results["rows_removed_nights_only"] = int(
    (before_df["minimum_nights"] > p95_nights).sum())
results["stats_after"] = df[["price", "minimum_nights"]].agg(
    ["mean", "median", "std", "max"]).round(2).to_dict()
results["iqr_outliers_after"] = {"price": iqr_outliers(df["price"]),
                                 "minimum_nights": iqr_outliers(df["minimum_nights"])}

# Chart 6 - histograms AFTER treatment
fig, axes = plt.subplots(1, 2, figsize=(9, 4.5))
sns.histplot(df["price"], bins=40, color=BLUE, ax=axes[0])
axes[0].set_title("Price")
axes[0].set_xlabel("Price per night (US dollars)")
axes[0].set_ylabel("Number of listings")
sns.histplot(df["minimum_nights"], bins=30, color=ORANGE, ax=axes[1])
axes[1].set_title("Minimum nights")
axes[1].set_xlabel("Minimum nights required")
axes[1].set_ylabel("Number of listings")
fig.suptitle("Chart 6: Price and Minimum Nights after Outlier Treatment")
save_chart("c6_after_hist.png")

# Chart 7 - boxplots AFTER treatment
fig, axes = plt.subplots(1, 2, figsize=(9, 4.5))
sns.boxplot(y=df["price"], ax=axes[0], color=BLUE)
axes[0].set_title("Price")
axes[0].set_ylabel("Price per night (US dollars)")
sns.boxplot(y=df["minimum_nights"], ax=axes[1], color=ORANGE)
axes[1].set_title("Minimum nights")
axes[1].set_ylabel("Minimum nights required")
```

```
fig.suptitle("Chart 7: Boxplots of Price and Minimum Nights (after treatment)")
save_chart("c7_boxplots_after.png")

# %% STEP 4 - FEATURE SELECTION AND CATEGORICAL ENCODING
# Task 4.1: high cardinality. How many different values does each have?
results["n_neighbourhood"] = int(df["neighbourhood"].nunique())
results["n_neighbourhood_group"] = int(df["neighbourhood_group"].nunique())
results["n_room_type"] = int(df["room_type"].nunique())
# One-hot encoding "neighbourhood" would add (unique values - 1) columns:
results["extra_columns_if_neighbourhood_encoded"] = results["n_neighbourhood"] - 1
df = df.drop(columns=["neighbourhood"])            # drop the very specific one

# Task 4.2: One-Hot Encoding with drop_first=True (avoids dummy variable trap)
df = pd.get_dummies(df, columns=["neighbourhood_group", "room_type"],
                    drop_first=True, dtype=int)
results["columns_after_encoding"] = list(df.columns)
results["shape_after_encoding"] = list(df.shape)

# %% STEP 5 - FEATURE-TARGET SEPARATION AND TRAIN-TEST SPLIT
# Task 5.1: X = features (inputs), y = target (what we predict)
X = df.drop(columns=["price"])
y = df["price"]
results["feature_names"] = list(X.columns)

# Task 5.2: 80% training, 20% testing, fixed seed for reproducibility
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42)
results["n_train"], results["n_test"] = int(len(X_train)), int(len(X_test))
results["train_mean_price"] = float(y_train.mean())
results["test_mean_price"] = float(y_test.mean())

# %% STEP 6 - LINEAR REGRESSION: TRAINING AND EVALUATION
model = LinearRegression()                  # Task 6.1: create the model
model.fit(X_train, y_train)                 # learn from the training data
y_pred = model.predict(X_test)              # Task 6.2: predict on unseen data
y_pred_train = model.predict(X_train)

# Task 6.3: the evaluation metrics
r2 = r2_score(y_test, y_pred)
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = float(np.sqrt(mse))
results["test_metrics"] = {"r2": float(r2), "mae": float(mae),
                           "mse": float(mse), "rmse": rmse}
results["train_metrics"] = {
    "r2": float(r2_score(y_train, y_pred_train)),
    "mae": float(mean_absolute_error(y_train, y_pred_train)),
    "rmse": float(np.sqrt(mean_squared_error(y_train, y_pred_train)))}
# Baseline: always guess the average training price
baseline = np.full(len(y_test), y_train.mean())
results["baseline_mae"] = float(mean_absolute_error(y_test, baseline))
results["baseline_rmse"] = float(np.sqrt(mean_squared_error(y_test, baseline)))
results["intercept"] = float(model.intercept_)
coef = pd.DataFrame({"feature": X.columns, "coefficient": model.coef_})
coef = coef.reindex(coef["coefficient"].abs().sort_values(
    ascending=False).index)
results["coefficients"] = coef.round(3).to_dict("records")

# %% STEP 7 - RESIDUAL ANALYSIS AND INTERPRETATION
residuals = y_test.values - y_pred          # residual = actual - predicted
results["residual_mean"] = float(residuals.mean())
results["residual_std"] = float(residuals.std())
results["within_25"] = float((np.abs(residuals) <= 25).mean() * 100)
results["within_50"] = float((np.abs(residuals) <= 50).mean() * 100)
results["pred_negative"] = int((y_pred < 0).sum())
results["pred_min"] = float(y_pred.min())
results["pred_max"] = float(y_pred.max())
results["residual_min"] = float(residuals.min())
results["residual_max"] = float(residuals.max())

# Task 7.1: Actual vs Predicted with a 45-degree reference line
plt.figure(figsize=(7, 6))
plt.scatter(y_test, y_pred, alpha=0.25, s=12, color=BLUE)
lims = [0, max(y_test.max(), y_pred.max())]
plt.plot(lims, lims, color=RED, lw=2, label="Perfect prediction (45-degree line)")
plt.xlim(lims); plt.ylim(min(0, y_pred.min()) - 10, lims[1])
plt.title("Chart 8: Actual vs Predicted Prices (test set)")
plt.xlabel("Actual price per night (US dollars)")
plt.ylabel("Predicted price per night (US dollars)")
plt.legend()
```

```
save_chart("c8_actual_vs_pred.png")

# Task 7.2a: residual plot (errors vs predicted values)
plt.figure(figsize=(8, 4.8))
plt.scatter(y_pred, residuals, alpha=0.25, s=12, color=BLUE)
plt.axhline(0, color=RED, lw=2)
plt.title("Chart 9: Residual Plot (Actual - Predicted vs Predicted)")
plt.xlabel("Predicted price per night (US dollars)")
plt.ylabel("Residual = actual - predicted (US dollars)")
save_chart("c9_residual_plot.png")

# Task 7.2b: error distribution histogram
plt.figure(figsize=(8, 4.5))
sns.histplot(residuals, bins=60, color=ORANGE)
plt.axvline(0, color=RED, lw=2)
plt.title("Chart 10: Distribution of Prediction Errors (Residuals)")
plt.xlabel("Residual = actual - predicted (US dollars)")
plt.ylabel("Number of test listings")
save_chart("c10_residual_hist.png")

# Task 7.3: cheap vs expensive listings - error by ACTUAL price band
bands = pd.cut(y_test.values, bins=[0, 75, 150, 250, p95_price],
               labels=["$1-75", "$76-150", "$151-250", f"$251-{p95_price:.0f}"])
res_df = pd.DataFrame({"band": bands, "actual": y_test.values,
                       "pred": y_pred, "resid": residuals})
res_df["abs_err"] = res_df["resid"].abs()
res_df["pct_err"] = res_df["abs_err"] / res_df["actual"] * 100
band_tbl = res_df.groupby("band", observed=True).agg(
    n=("actual", "size"), mean_actual=("actual", "mean"),
    mean_pred=("pred", "mean"), mae=("abs_err", "mean"),
    bias=("resid", "mean"), median_pct_err=("pct_err", "median"),
    mean_pct_err=("pct_err", "mean")).round(2)
results["band_table"] = band_tbl.reset_index().astype({"band": str}
                                                      ).to_dict("records")
# Error by room type (extra insight)
room_cols = [c for c in X_test.columns if c.startswith("room_type_")]
rt = np.where(X_test["room_type_Private room"] == 1, "Private room",
              np.where(X_test["room_type_Shared room"] == 1, "Shared room",
                       "Entire home/apt"))
rt_df = pd.DataFrame({"room": rt, "abs_err": np.abs(residuals),
                      "resid": residuals})
results["room_error"] = rt_df.groupby("room").agg(
    n=("abs_err", "size"), mae=("abs_err", "mean"),
    bias=("resid", "mean")).round(2).reset_index().to_dict("records")

# Chart 11 - average error by price band
plt.figure(figsize=(8, 4.5))
x = np.arange(len(band_tbl))
plt.bar(x - 0.2, band_tbl["mae"], width=0.4, color=BLUE, label="Average error (MAE, $)")
plt.bar(x + 0.2, band_tbl["bias"], width=0.4, color=ORANGE,
        label="Average residual (actual - predicted, $)")
plt.axhline(0, color="black", lw=1)
plt.xticks(x, band_tbl.index.astype(str))
plt.title("Chart 11: Prediction Error by Actual Price Band")
plt.xlabel("Actual nightly price band (US dollars)")
plt.ylabel("US dollars")
plt.legend()
save_chart("c11_error_by_band.png")

# %% STEP 8 - SAVE RESULTS (used to build the report)
with open("results.json", "w") as f:
    json.dump(results, f, indent=1, default=str)
print("R2   :", round(r2, 4))
print("MAE  :", round(mae, 2))
print("MSE  :", round(mse, 2))
print("RMSE :", round(rmse, 2))
print("Analysis finished. Charts saved in:", OUT_DIR)
```